

CURSO DE SQL Y BASES DE DATOS RELACIONALES

SQL

Inserción, Consulta y Administración de información en bases de
datos relacionales basadas en SQL

Material de Referencia

miyoshikamium@gmail.mx

Versión 5.0 — 8 de julio de 2026

Resumen

Este documento constituye el material teórico de referencia para el Módulo 1 del *Curso de SQL y Bases de Datos Relacionales*. Se abordan los fundamentos históricos y conceptuales que dieron origen a los sistemas de gestión de bases de datos (SGBD) relacionales, la evolución del lenguaje SQL desde sus orígenes en los laboratorios de IBM hasta los estándares modernos, y los principios del modelo relacional propuesto por Edgar F. Codd.

El objetivo es proveer al estudiante una base conceptual sólida que permita contextualizar cada construcción sintáctica de SQL dentro de un marco teórico coherente, facilitando un aprendizaje significativo y duradero. El curso progresa desde la consulta básica de datos, pasando por la modificación y escritura, hasta la administración de esquemas y la integridad referencial.

Palabras clave: base de datos, modelo relacional, SQL, SGBD, álgebra relacional, normalización, ACID, DML, DDL.

Índice

1. El Problema de los Datos: Una Perspectiva Histórica	3
1.1. Era pre-relacional (1950–1970)	3
2. Edgar F. Codd y el Modelo Relacional	4
2.1. Las 12 reglas de Codd	4
3. Nacimiento de SQL: De SEQUEL a SQL	5
3.1. El primer sistema comercial: Oracle	5
4. ¿Qué es una Base de Datos?	6
4.1. Analogía con el mundo real	6

5. Administración de la Base de Datos	7
5.1. CREATE DATABASE	7
5.2. DROP DATABASE	8
6. Administración de Tablas	9
6.1. CREATE TABLE	9
6.2. Restricciones de Integridad	11
6.2.1. NOT NULL	11
6.2.2. UNIQUE	11
6.2.3. PRIMARY KEY	12
6.2.4. CHECK	13
6.2.5. FOREIGN KEY	13
6.2.6. DEFAULT	14
6.2.7. AUTO_INCREMENT	14
6.3. DROP TABLE	15
6.4. ALTER TABLE	15
6.4.1. ADD	16
6.4.2. RENAME COLUMN	16
6.4.3. MODIFY COLUMN	17
6.4.4. DROP COLUMN	17
7. Consulta de Datos: SELECT	19
7.1. Sintaxis completa y cláusulas	19
7.2. Tabla de ejemplo	20
7.3. Proyección de columnas: SELECT * y columnas específicas	20
7.4. Operadores aritméticos	21
7.5. Operadores de comparación	22
8. Modificadores: Filtrado, Ordenamiento y Agrupación	23
8.1. DISTINCT	23
8.2. WHERE	23
8.3. ORDER BY	24
8.4. LIKE	24
8.5. AND, OR, NOT	25
8.6. IN y BETWEEN	26
8.7. GROUP BY y HAVING	26
8.8. CASE	28
8.9. COALESCE e IFNULL	28
8.10. Combinación de tablas: JOIN	28

8.10.1. INNER JOIN	29
8.10.2. LEFT JOIN	29
8.10.3. RIGHT JOIN	30
8.10.4. UNION	30
9. Escritura de Datos	32
9.1. INSERT	32
9.2. UPDATE	33
9.3. DELETE	34
9.3.1. Diferencias entre DELETE y TRUNCATE	34
10. Almacenamiento de Datos Relacionados	35
10.1. Inserción en Tablas con Relación 1:1	35
10.2. Inserción en Tablas con Relación 1:N	36
10.3. Inserción en Tablas con Relación N:M	36

1. El Problema de los Datos: Una Perspectiva Histórica

Desde los albores de la civilización, la humanidad ha necesitado almacenar y recuperar información de manera sistemática. Las tablillas de arcilla sumerias (3000 a.C.), los registros contables del Imperio Romano, o los libros de contabilidad de los comerciantes medievales son, en esencia, *sistemas de gestión de información*: colecciones de datos organizados con un propósito definido y un mecanismo de consulta.

La revolución informática de mediados del siglo XX transformó este problema milenario en una disciplina formal. Con la proliferación de computadoras en organizaciones bancarias, gubernamentales y empresariales durante los años 1950 y 1960, la necesidad de almacenar grandes volúmenes de datos de forma eficiente y recuperarlos con rapidez se volvió crítica.

1.1. Era pre-relacional (1950–1970)

Los primeros sistemas de gestión de datos surgieron como soluciones ad-hoc a problemas específicos. Predominaban dos modelos principales:

- **Modelo jerárquico:** Los datos se organizaban en una estructura de árbol. IBM desarrolló IMS (*Information Management System*) en 1966 para la misión Apolo de la NASA **date2003introduction**. Aunque eficiente para consultas predefinidas, era rígido: agregar una nueva relación entre datos requería reestructurar físicamente el sistema.
- **Modelo de red (CODASYL):** Propuesto por el comité CODASYL en 1969, per-

mitía relaciones muchos-a-muchos mediante punteros físicos entre registros **codasy11971**. Era más flexible que el jerárquico, pero extremadamente complejo de administrar y altamente dependiente de la estructura física del almacenamiento.

El problema fundamental de ambos modelos era la **dependencia físico-lógica**: los programas de aplicación debían conocer cómo estaban organizados físicamente los datos en disco para poder acceder a ellos. Un cambio en la estructura de almacenamiento implicaba reescribir todos los programas.

2. Edgar F. Codd y el Modelo Relacional

En junio de 1970, Edgar Frank Codd —matemático e investigador de IBM— publicó el artículo que cambiaría para siempre el mundo de las bases de datos: “*A Relational Model of Data for Large Shared Data Banks*” en la revista *Communications of the ACM* **codd1970**.

Definición: Modelo Relacional Codd

Un modelo de datos en el que la información se representa exclusivamente mediante **relaciones** (tablas bidimensionales), y en el que todas las operaciones sobre los datos se expresan mediante un conjunto cerrado de operaciones del **álgebra relacional**, sin necesidad de conocer la organización física del almacenamiento.

La contribución de Codd fue revolucionaria por tres razones:

1. **Independencia de datos:** Los programas de aplicación operan sobre una representación lógica (tablas), completamente desacoplada de cómo los datos se almacenan físicamente.
2. **Fundamento matemático:** El modelo se basa en la teoría de conjuntos y la lógica de predicados de primer orden, lo que permite razonar formalmente sobre las operaciones.
3. **Lenguaje declarativo:** El programador especifica *qué* datos quiere, no *cómo* obtenerlos. El sistema optimiza el acceso automáticamente.

2.1. Las 12 reglas de Codd

En 1985, Codd formalizó su modelo en 12 reglas (numeradas del 0 al 12) que debe cumplir un sistema para ser considerado verdaderamente relacional **codd1985**. Las más relevantes para este curso son:

Regla	Descripción
0	Gestión mediante capacidades relacionales
1	Representación de la información como tablas
2	Acceso garantizado a cada dato (tabla + clave + columna)
3	Tratamiento sistemático de valores nulos
6	Actualización de vistas
9	Independencia de datos física y lógica
12	No subversión del modelo relacional

Tabla 1: Selección de las 12 reglas de Codd (1985).

3. Nacimiento de SQL: De SEQUEL a SQL

A raíz de la publicación de Codd, IBM inició el proyecto *System R* en 1973 en sus laboratorios de San José, California, con el objetivo de demostrar la viabilidad práctica del modelo relacional **chamberlin**1974. Donald D. Chamberlin y Raymond F. Boyce diseñaron un lenguaje de consulta denominado **SEQUEL** (*Structured English Query Language*).

El nombre fue posteriormente acortado a **SQL** (*Structured Query Language*) por razones legales de marca registrada. En paralelo, Michael Stonebraker y Eugene Wong desarrollaron **INGRES** en la Universidad de California, Berkeley (1973–1977), que introdujo su propio lenguaje de consulta llamado QUEL.

Nota importante

Aunque coloquialmente se pronuncia tanto “sequel” como “ese-cu-ele”, el estándar ISO/IEC lo denomina formalmente “SQL”. En este curso usaremos la pronunciación “ese-cu-ele” por su precisión técnica.

3.1. El primer sistema comercial: Oracle

En 1979, la empresa Relational Software Inc. (hoy Oracle Corporation), fundada por Larry Ellison, liberó el primer sistema de gestión de bases de datos relacionales comercial: **Oracle V2**. Paradójicamente, Oracle llegó al mercado antes que el propio IBM con su DB2 (1983), a pesar de que la tecnología subyacente provenía de los laboratorios de IBM.

4. ¿Qué es una Base de Datos?

Definición: Base de Datos

Una **base de datos** es una colección organizada, persistente y compartible de datos relacionados entre sí, diseñada para minimizar la redundancia y maximizar la integridad, de modo que múltiples aplicaciones y usuarios puedan acceder a ella de manera concurrente y controlada **date2003introduction**.

Esta definición merece descomponerse en sus atributos esenciales:

- **Organizada:** Los datos siguen una estructura formal (esquema) que define qué tipos de información se almacenan y cómo se relacionan entre sí.
- **Persistente:** Los datos sobreviven a la terminación de los procesos que los crean. A diferencia de las variables en memoria, los datos en una base de datos permanecen aunque el sistema se apague.
- **Compartible:** Múltiples usuarios o aplicaciones pueden acceder a los mismos datos simultáneamente, de forma controlada y sin interferencias.
- **Mínima redundancia:** El mismo dato no se repite innecesariamente en múltiples lugares, lo que reduce el riesgo de inconsistencias (un dato cambia en un lugar pero no en otro).

4.1. Analogía con el mundo real

Imaginemos el sistema de una biblioteca universitaria:

Ejemplo práctico

Una biblioteca almacena información sobre **libros, estudiantes y préstamos**. Sin una base de datos, cada ficha de préstamo copiaría el título completo del libro, el autor, el nombre del estudiante, su carrera, etc. Si un estudiante cambia su dirección, habría que actualizar decenas de fichas físicas. Con una base de datos relacional, el estudiante aparece *una sola vez* en la tabla **Estudiantes**. Todos los préstamos simplemente hacen referencia a su identificador. Un cambio de dirección se realiza en un único lugar y se refleja automáticamente en todo el sistema.

5. Administración de la Base de Datos

Las sentencias de administración de bases de datos pertenecen al sublenguaje **DDL** (*Data Definition Language*). Operan sobre el contenedor de más alto nivel en MySQL: la **base de datos** (también llamada *schema*), que agrupa tablas, vistas, procedimientos y demás objetos.

5.1. CREATE DATABASE

Definición: CREATE DATABASE

Crea una nueva base de datos vacía en el servidor. Permite especificar el **juego de caracteres** (*character set*) y la **colación** (*collation*) que determinarán cómo se almacenan y comparan las cadenas de texto en todas sus tablas.

```
1 CREATE DATABASE [IF NOT EXISTS] nombre_bd
2 [CHARACTER SET charset_name]
3 [COLLATE collation_name];
```

Listing 1: Sintaxis de CREATE DATABASE

Ejemplo práctico

Crear una base de datos para una aplicación en español con soporte completo de Unicode:

```
1 CREATE DATABASE IF NOT EXISTS tienda_online
2 CHARACTER SET utf8mb4
3 COLLATE utf8mb4_unicode_ci;
4
5 -- Seleccionar la base de datos activa
6 USE tienda_online;
```

Listing 2: Base de datos con utf8mb4 y collation en español

Juegos de caracteres y colaciones más comunes:

Tabla 2: Character sets y collations habituales en MySQL

charset	collation	Uso recomendado
utf8mb4	utf8mb4_unicode_ci	Unicode completo; soporta emojis; recomendado
utf8mb4	utf8mb4_0900_ai_ci	Unicode 9.0; sin acento/mayúsculas; MySQL 8.0+
utf8mb4	utf8mb4_bin	Comparación binaria; distingue mayúsculas
latin1	latin1_swedish_ci	Legado; solo caracteres ISO-8859-1

Nota importante

Usar `utf8` (sin `mb4`) en MySQL es un error frecuente: ese alias solo soporta hasta 3 bytes por carácter y excluye emojis y algunos ideogramas CJK. La elección correcta para proyectos nuevos es siempre `utf8mb4`.

```
1 SHOW DATABASES;                -- listar todas las
    bases de datos
2 SHOW CREATE DATABASE tienda_online; -- ver la definici n
    completa
3 SELECT DATABASE();              -- ver la base de datos
    activa
```

Listing 3: Comandos de exploración de bases de datos

5.2. DROP DATABASE

Definición: DROP DATABASE

Elimina la base de datos y **todos los objetos que contiene** (tablas, vistas, procedimientos, datos) de forma permanente e irreversible. No existe confirmación ni papelera de reciclaje.

```
1 DROP DATABASE [IF EXISTS] nombre_bd;
```

Listing 4: Sintaxis de DROP DATABASE

Ejemplo práctico

```
1 -- Genera error si la base de datos no existe
2 DROP DATABASE tienda_online;
3
4 -- No genera error si no existe ( til en scripts de
    despliegue)
5 DROP DATABASE IF EXISTS tienda_online;
```

Listing 5: DROP DATABASE con y sin guarda

Nota importante

Precauciones antes de ejecutar DROP DATABASE:

1. Verificar que se tiene un respaldo (*backup*) actualizado.
2. Confirmar que ninguna aplicación activa tiene conexiones abiertas a esa base de datos.
3. Usar siempre `IF EXISTS` en scripts de producción.

6. Administración de Tablas

Las sentencias de administración de tablas pertenecen al sublenguaje **DDL** (*Data Definition Language*). A diferencia de las instrucciones DML, los cambios DDL son **autoconfirmados** (*auto-commit*) en la mayoría de los motores: no pueden deshacerse con ROLLBACK.

6.1. CREATE TABLE

Definición: CREATE TABLE

Crea una nueva tabla en la base de datos activa. La sentencia define el nombre de cada columna, su **tipo de dato**, las **restricciones** de integridad y las **opciones de tabla** (motor de almacenamiento, juego de caracteres, etc.).

Sintaxis completa:

```
1 CREATE TABLE [IF NOT EXISTS] nombre_tabla (  
2 columna1 tipo [opciones_columna] [restriccion_columna],  
3 columna2 tipo [opciones_columna] [restriccion_columna],  
4 ...  
5 [restricciones_tabla]  
6 ) [opciones_tabla];
```

Listing 6: Estructura general de CREATE TABLE

Tipos de datos más utilizados en MySQL:

Tabla 3: Tipos de datos frecuentes en MySQL

Tipo	Categoría	Descripción
TINYINT	Entero	1 byte; rango -128 a 127 (o $0-255$ UNSIGNED)
INT / INTEGER	Entero	4 bytes; rango $\pm 2,1 \times 10^9$
BIGINT	Entero	8 bytes; para identificadores de gran volumen
DECIMAL(p,s)	Decimal	Precisión exacta; ideal para valores monetarios
FLOAT / DOUBLE	Flotante	Precisión aproximada; uso científico
CHAR(n)	Texto	Longitud fija n ; rellena con espacios
VARCHAR(n)	Texto	Longitud variable hasta n caracteres
TEXT	Texto	Hasta 65 535 bytes; sin valor DEFAULT
DATE	Fecha	Formato YYYY-MM-DD
DATETIME	Fecha/hora	Formato YYYY-MM-DD HH:MM:SS
TIMESTAMP	Fecha/hora	UTC; se actualiza automáticamente con ON UPDATE
BOOLEAN / TINYINT(1)	Lógico	MySQL almacena booleanos como TINYINT(1)
ENUM('a','b',...)	Catálogo	Solo acepta los valores de la lista

Ejemplo práctico

Tabla completa con varios tipos de datos, restricciones y opciones:

```

1 CREATE TABLE IF NOT EXISTS productos (
2   producto_id INT UNSIGNED NOT NULL AUTO_INCREMENT,
3   sku VARCHAR(30) NOT NULL,
4   nombre VARCHAR(120) NOT NULL,
5   descripcion TEXT,
6   precio DECIMAL(10,2) NOT NULL DEFAULT 0.00,
7   stock INT UNSIGNED NOT NULL DEFAULT 0,
8   activo TINYINT(1) NOT NULL DEFAULT 1,
9   categoria_id INT UNSIGNED NOT NULL,
10  creado_en DATETIME NOT NULL DEFAULT
      CURRENT_TIMESTAMP,
11  actualizado TIMESTAMP NOT NULL
12  DEFAULT CURRENT_TIMESTAMP
13  ON UPDATE CURRENT_TIMESTAMP,
14
15  PRIMARY KEY (producto_id),
16  UNIQUE KEY uq_sku (sku),
17  CONSTRAINT chk_precio CHECK (precio >= 0),
18  CONSTRAINT fk_prod_cat

```

```

19 FOREIGN KEY (categoria_id)
20 REFERENCES categorias(categoria_id)
21 ON DELETE RESTRICT
22 ON UPDATE CASCADE
23 )
24 ENGINE = InnoDB
25 CHARSET = utf8mb4
26 COLLATE = utf8mb4_unicode_ci;

```

Listing 7: CREATE TABLE con opciones reales de producción

6.2. Restricciones de Integridad

6.2.1. NOT NULL

Definición: NOT NULL

Prohíbe que una columna almacene el valor **NULL**. Toda inserción o actualización que no suministre un valor para esa columna — y que no tenga **DEFAULT** definido — será rechazada por el motor.

```

1 CREATE TABLE empleados (
2   empleado_id INT NOT NULL AUTO_INCREMENT,
3   nombre VARCHAR(100) NOT NULL,      -- obligatorio
4   telefono VARCHAR(20) NULL,         -- opcional (admite
      NULL)
5   correo VARCHAR(120) NOT NULL,
6   PRIMARY KEY (empleado_id)
7 );

```

Listing 8: Columnas con y sin NOT NULL

Nota importante

Omitir **NOT NULL** equivale implícitamente a **NULL** en MySQL. Declararlo de forma explícita mejora la legibilidad y la intención del esquema.

6.2.2. UNIQUE

Definición: UNIQUE

Garantiza que todos los valores de una columna (o combinación de columnas) sean distintos dentro de la tabla. A diferencia de **PRIMARY KEY**, una tabla puede tener **múltiples** restricciones **UNIQUE** y estas **sí permiten** el valor **NULL** (con matices según el motor).

```

1 CREATE TABLE usuarios (

```

```

2 usuario_id INT UNSIGNED AUTO_INCREMENT PRIMARY
  KEY,
3 correo VARCHAR(120) NOT NULL UNIQUE, -- nivel
  columna
4 username VARCHAR(50) NOT NULL,
5 doc_tipo CHAR(3) NOT NULL,
6 doc_numero VARCHAR(20) NOT NULL,
7
8 -- UNIQUE compuesto a nivel de tabla
9 UNIQUE KEY uq_documento (doc_tipo, doc_numero)
10 );

```

Listing 9: UNIQUE a nivel de columna y de tabla

Tabla 4: Diferencias entre UNIQUE y PRIMARY KEY

Característica	PRIMARY KEY	UNIQUE
Cantidad por tabla	Solo una	Múltiples
Admite NULL	No	Sí (un NULL por columna en MySQL)
Índice generado	Clúster (InnoDB)	Secundario no clúster
Uso habitual	Identificador de fila	Atributos alternativos únicos

6.2.3. PRIMARY KEY

Definición: PRIMARY KEY

Identifica de forma **única e irrepitable** cada fila de la tabla. Implica NOT NULL y UNIQUE de forma automática. Puede ser **simple** (una columna) o **compuesta** (varias columnas).

```

1 -- Clave primaria simple (columna nica )
2 CREATE TABLE categorias (
3 categoria_id INT UNSIGNED AUTO_INCREMENT,
4 nombre VARCHAR(80) NOT NULL,
5 PRIMARY KEY (categoria_id)
6 );
7
8 -- Clave primaria compuesta (tabla intermedia N:M)
9 CREATE TABLE inscripciones (
10 estudiante_id INT UNSIGNED NOT NULL,
11 curso_id INT UNSIGNED NOT NULL,
12 fecha_alta DATE NOT NULL,
13 PRIMARY KEY (estudiante_id, curso_id) -- par debe ser
  nico
14 );

```

Listing 10: PK simple vs. PK compuesta

Nota importante

En InnoDB, la PRIMARY KEY determina el **índice clúster**: las filas se almacenan físicamente ordenadas por ella. Elegir una PK de tipo entero secuencial (AUTO_INCREMENT) produce inserciones más eficientes que una PK de tipo cadena o UUID aleatorio.

6.2.4. CHECK**Definición: CHECK**

Valida que el valor de una columna cumpla una **expresión booleana** antes de aceptar la inserción o actualización. Si la expresión devuelve FALSE, la operación es rechazada. Disponible en MySQL a partir de la versión 8.0.16.

```

1 CREATE TABLE contratos (
2   contrato_id    INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
3   fecha_inicio  DATE          NOT NULL,
4   fecha_fin      DATE,
5   salario        DECIMAL(10,2) NOT NULL,
6   CONSTRAINT chk_fecha CHECK (fecha_fin > fecha_inicio),
7   CONSTRAINT chk_salario CHECK (salario > 0)
8 );

```

Listing 11: CHECK a nivel de columna y de tabla

6.2.5. FOREIGN KEY**Definición: FOREIGN KEY**

Establece una **integridad referencial**: la columna o combinación de columnas debe coincidir con valores que existen en la clave primaria de otra tabla (la tabla referenciada). Las acciones ON DELETE y ON UPDATE especifican qué sucede cuando se modifica la tabla padre.

```

1 CREATE TABLE pedidos (
2   pedido_id      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
3   cliente_id     INT UNSIGNED NOT NULL,
4   fecha          DATE NOT NULL,
5   FOREIGN KEY (cliente_id) REFERENCES clientes(cliente_id)
6   ON DELETE RESTRICT      -- no permite eliminar si hay
                           pedidos
7   ON UPDATE CASCADE      -- actualiza el ID si cambia en
                           clientes
8 );

```

Listing 12: FOREIGN KEY con políticas de actualización

6.2.6. DEFAULT

Definición: DEFAULT

Asigna un valor automático a una columna cuando no se proporciona uno durante una inserción. Acelera la entrada de datos y reduce valores NULL innecesarios.

```
1  -- DEFAULT con valores est ticos
2  CREATE TABLE usuarios (
3  usuario_id    INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
4  activo        TINYINT(1) DEFAULT 1,
5  saldo         DECIMAL(10,2) DEFAULT 0.00,
6  fecha_registro DATETIME DEFAULT CURRENT_TIMESTAMP
7  );
```

Listing 13: DEFAULT con valores estáticos y dinámicos

6.2.7. AUTO_INCREMENT

Definición: AUTO_INCREMENT

Genera automáticamente valores enteros secuenciales para una columna en cada nueva inserción. Solo puede haber un AUTO_INCREMENT por tabla y típicamente se usa con PRIMARY KEY.

```
1  CREATE TABLE productos (
2  producto_id  INT UNSIGNED NOT NULL AUTO_INCREMENT,
3  nombre       VARCHAR(120) NOT NULL,
4  PRIMARY KEY (producto_id)
5  );
6
7  -- El valor inicial es 1 por defecto
8  ALTER TABLE productos AUTO_INCREMENT = 1000;  -- cambiar
           inicio
```

Listing 14: AUTO_INCREMENT en contexto

Nota importante

El contador AUTO_INCREMENT **nunca retrocede** al eliminar filas. Si se inserta y luego se borra el registro con ID = 1005, el siguiente ID será 1006, no 1005. Esto es intencional para evitar colisiones en entornos concurrentes.

6.3. DROP TABLE

Definición: DROP TABLE

Elimina una tabla de forma **permanente e irreversible**: borra la estructura, todos sus datos y los índices asociados. No puede deshacerse con ROLLBACK.

```

1  -- Falla con error si la tabla no existe
2  DROP TABLE pedidos;
3
4  -- No genera error si la tabla no existe
5  DROP TABLE IF EXISTS pedidos;
6
7  -- Eliminar varias tablas en una sola sentencia
8  DROP TABLE IF EXISTS inscripciones, pedidos, sesiones;
```

Listing 15: DROP TABLE básico y con guarda

Orden correcto con claves foráneas:

```

1  -- ERROR: no se puede eliminar 'clientes' mientras 'pedidos
   ' la referencia
2  DROP TABLE clientes;    -- falla
3
4  -- Correcto: eliminar primero la tabla HIJO, luego la tabla
   PADRE
5  DROP TABLE IF EXISTS pedidos;    -- hijo (tiene la FK)
6  DROP TABLE IF EXISTS clientes;    -- padre (tiene la PK
   referenciada)
```

Listing 16: DROP TABLE respetando integridad referencial

Nota importante

Como alternativa al orden manual, es posible deshabilitar temporalmente la comprobación de claves foráneas. Esta práctica debe usarse con extrema cautela y solo en scripts de mantenimiento controlados:

```

1  SET FOREIGN_KEY_CHECKS = 0;
2  DROP TABLE IF EXISTS clientes, pedidos;
3  SET FOREIGN_KEY_CHECKS = 1;
```

Listing 17: Deshabilitar FK solo para scripts de mantenimiento

6.4. ALTER TABLE

Definición: ALTER TABLE

Modifica la **estructura** de una tabla existente sin necesidad de recrearla. Permite agregar, renombrar, modificar y eliminar columnas, así como gestionar restricciones e índices.

6.4.1. ADD

Agrega nuevas columnas o restricciones a una tabla existente.

```
1  -- Agregar una columna al final (comportamiento por defecto
   )
2  ALTER TABLE productos
3  ADD COLUMN descuento DECIMAL(5,2) NOT NULL DEFAULT 0.00;
4
5  -- Agregar una columna en una posición específica
6  ALTER TABLE productos
7  ADD COLUMN imagen_url VARCHAR(255) NULL
8  AFTER nombre;
9
10 -- Agregar al principio
11 ALTER TABLE productos
12 ADD COLUMN prioridad TINYINT NOT NULL DEFAULT 0
13 FIRST;
```

Listing 18: ADD COLUMN — agregar columnas

```
1  -- Agregar índice UNIQUE
2  ALTER TABLE usuarios
3  ADD CONSTRAINT uq_correo UNIQUE (correo);
4
5  -- Agregar clave foránea
6  ALTER TABLE pedidos
7  ADD CONSTRAINT fk_pedido_cliente
8  FOREIGN KEY (cliente_id) REFERENCES clientes(cliente_id)
9  ON DELETE RESTRICT ON UPDATE CASCADE;
10
11 -- Agregar CHECK
12 ALTER TABLE productos
13 ADD CONSTRAINT chk_descuento CHECK (descuento BETWEEN 0 AND
   100);
```

Listing 19: ADD CONSTRAINT — agregar restricciones

6.4.2. RENAME COLUMN

Cambia el nombre de una columna preservando su tipo, restricciones y datos. Disponible en MySQL 8.0+.

```
1  -- Renombrar una columna (MySQL 8.0+)
2  ALTER TABLE clientes
3  RENAME COLUMN telefono TO telefono_principal;
4
5  -- Renombrar varias columnas en una sola sentencia
6  ALTER TABLE productos
7  RENAME COLUMN img_url TO imagen_url,
8  RENAME COLUMN desc_prod TO descripcion;
```

Listing 20: RENAME COLUMN

Nota importante

En versiones anteriores a MySQL 8.0, el renombrado se realiza con `CHANGE COLUMN`, que requiere repetir el tipo de dato:

```
1 ALTER TABLE clientes
2 CHANGE COLUMN telefono telefono_principal VARCHAR(20)
  NULL;
```

Listing 21: CHANGE COLUMN para MySQL anterior a 8.0

6.4.3. MODIFY COLUMN

Cambia el **tipo de dato**, las restricciones o las opciones de una columna existente sin alterar su nombre.

```
1 -- Ampliar el tamaño de un VARCHAR
2 ALTER TABLE productos
3 MODIFY COLUMN nombre VARCHAR(200) NOT NULL;
4
5 -- Cambiar tipo y agregar restricción
6 ALTER TABLE contratos
7 MODIFY COLUMN salario DECIMAL(12,2) NOT NULL DEFAULT 0.00;
8
9 -- Cambiar posición de la columna
10 ALTER TABLE empleados
11 MODIFY COLUMN correo VARCHAR(120) NOT NULL AFTER nombre;
```

Listing 22: MODIFY COLUMN — cambios de tipo y propiedades

Nota importante

`MODIFY COLUMN` reescribe la definición completa de la columna. Omitir `NOT NULL` o `DEFAULT` los elimina, aunque el dato ya exista. Siempre verificar la definición actual con `DESCRIBE` antes de modificar.

6.4.4. DROP COLUMN

Elimina una columna y **todos sus datos** de forma permanente.

```
1 -- Eliminar una columna
2 ALTER TABLE productos
3 DROP COLUMN imagen_url;
4
5 -- Eliminar varias columnas en una sola sentencia
6 ALTER TABLE empleados
7 DROP COLUMN fax,
8 DROP COLUMN extension;
9
10 -- Eliminar una restricción antes de poder borrar la
    columna
```

```
11 ALTER TABLE pedidos
12 DROP FOREIGN KEY fk_pedido_cliente,
13 DROP COLUMN cliente_id;
```

Listing 23: DROP COLUMN

Nota importante

Si la columna forma parte de un índice o restricción, MySQL exige eliminar primero esa dependencia. Usar `SHOW CREATE TABLE` para identificar los nombres exactos de índices y claves foráneas antes de ejecutar el `DROP`.

7. Consulta de Datos: SELECT

La instrucción **SELECT** es, sin duda, la más utilizada y la más rica en posibilidades de todo el lenguaje SQL. Su propósito es **consultar** datos: extraer filas de una o varias tablas, aplicarles filtros, transformaciones y ordenamientos, y devolver un resultado tabular. A diferencia de **INSERT**, **UPDATE** o **DELETE**, **SELECT** es una operación de **sólo lectura**: nunca modifica los datos almacenados.

7.1. Sintaxis completa y cláusulas

La sintaxis formal de **SELECT** en MySQL/SQL estándar es la siguiente. Las partes entre corchetes [] son opcionales; las palabras en mayúsculas son palabras reservadas de SQL:

```

1 SELECT [ALL | DISTINCT | DISTINCTROW]
2 lista_de_columnas | *
3 FROM nombre_tabla [[AS] alias_tabla]
4 [WHERE condicion]
5 [GROUP BY columna [ASC | DESC], ...]
6 [HAVING condicion_de_grupo]
7 [ORDER BY columna [ASC | DESC], ...]
8 [LIMIT numero_de_filas [OFFSET desplazamiento]];
```

Listing 24: Sintaxis completa de **SELECT**.

Cada cláusula tiene un rol preciso dentro de la consulta. La table 5 resume su función y el orden en que el SGBD las *evalúa* internamente (que no siempre coincide con el orden en que se escriben):

Cláusula	Orden eval.	Función
FROM	1	Determina la(s) tabla(s) fuente de datos.
WHERE	2	Filtra filas <i>antes</i> de agrupar.
GROUP BY	3	Agrupar filas con valores iguales en las columnas indicadas.
HAVING	4	Filtra grupos <i>después</i> de agrupar.
SELECT	5	Proyecta (selecciona) las columnas del resultado.
DISTINCT	6	Elimina filas duplicadas del resultado proyectado.
ORDER BY	7	Ordena el resultado final.
LIMIT	8	Recorta el número de filas devueltas.

Tabla 5: Cláusulas de **SELECT** y su orden de evaluación interno.

Nota importante

El orden de **escritura** de las cláusulas es fijo y debe respetarse exactamente como aparece en la sintaxis de arriba. Sin embargo, el SGBD las **evalúa** en un orden diferente (véase table 5). Esto explica, por ejemplo, por qué no se puede

usar un alias definido en **SELECT** dentro de la cláusula **WHERE**: **WHERE** se evalúa *antes* de que exista el alias.

7.2. Tabla de ejemplo

Para ilustrar todos los ejemplos de esta sección usaremos una tabla llamada **empleados** con la siguiente estructura y datos:

```

1 CREATE TABLE empleados (
2   id            INT            PRIMARY KEY AUTO_INCREMENT,
3   nombre        VARCHAR(60)    NOT NULL,
4   departamento  VARCHAR(40)    NOT NULL,
5   cargo         VARCHAR(40)    NOT NULL,
6   salario       DECIMAL(10,2)  NOT NULL,
7   fecha_ingreso DATE           NOT NULL
8 );
9
10 INSERT INTO empleados (nombre, departamento, cargo, salario
11   , fecha_ingreso) VALUES
12 ('Ana Garc a',      'Ventas',      'Gerente',      55000.00,
13   '2018-03-15'),
14 ('Luis Herrera',    'Ventas',      'Analista',     32000.00, '
15   2020-07-01'),
16 ('Sof a Ram rez',   'TI',          'Desarrolladora',
17   48000.00, '2019-11-20'),
18 ('Carlos Mendoza', 'TI',          'DBA',          51000.00, '
19   2017-05-10'),
20 ('Elena Torres',    'RRHH',        'Coordinadora', 38000.00,
   '2021-02-28'),
   ('Marco Villanueva', 'Ventas',      'Analista',     31000.00, '
   2022-09-05'),
   ('Patricia Luna',    'TI',          'Desarrolladora',
   47000.00, '2020-04-14'),
   ('Roberto Salas',    'RRHH',        'Gerente',      52000.00, '
   2016-08-22'),
   ('Diana Castillo',  'Finanzas',    'Analista',     39000.00, '
   2023-01-10'),
   ('Javier Moreno',   'Finanzas',    'Gerente',      58000.00, '
   2015-06-30');

```

Listing 25: Creación e inicialización de la tabla **empleados**.

7.3. Proyección de columnas: **SELECT *** y columnas específicas

La **proyección** es la operación de elegir qué columnas aparecerán en el resultado. El asterisco ***** es un comodín que significa “*todas las columnas*”.

```

1 -- Devuelve todas las columnas de todos los empleados

```

```

2 SELECT *
3 FROM empleados;

```

Listing 26: Seleccionar todas las columnas.

```

1 -- S lo nombre, departamento y salario
2 SELECT nombre, departamento, salario
3 FROM empleados;

```

Listing 27: Seleccionar columnas específicas.

Nota importante

En producción se recomienda **evitar** `SELECT *` porque: (1) transfiere columnas innecesarias por la red, (2) si alguien agrega una columna sensible a la tabla, la consulta la expondrá automáticamente, (3) dificulta la legibilidad del código. Úsalo sólo para exploración interactiva.

También es posible incluir **expresiones** en la lista de columnas: literales, operaciones aritméticas o llamadas a funciones:

```

1 SELECT
2 nombre,
3 salario,
4 salario * 12          AS salario_anual,
5 salario * 0.10        AS bono_estimado,
6 'Activo'              AS estado    -- literal de texto
7 FROM empleados;

```

Listing 28: Columnas calculadas y literales en SELECT.

7.4. Operadores aritméticos

Dentro de `SELECT` (y también en `WHERE`) se pueden usar operadores aritméticos estándar sobre columnas numéricas:

Op.	Nombre	Ejemplo
+	Suma	<code>salario + 5000</code>
-	Resta	<code>salario - descuento</code>
*	Multiplicación	<code>salario * 1.15</code>
/	División	<code>salario / 14</code>
%	Módulo (resto)	<code>id % 2</code> (par/impar)

Tabla 6: Operadores aritméticos en SQL.

```

1 SELECT
2 nombre,
3 salario,

```

```

4 salario * 1.15      AS salario_con_aumento,
5 (salario / 30)      AS salario_diario,
6 salario - 5000      AS salario_base
7 FROM empleados
8 WHERE departamento = 'TI';

```

Listing 29: Uso de operadores aritméticos.

7.5. Operadores de comparación

Los operadores de comparación se usan principalmente en la cláusula WHERE para filtrar filas. Devuelven un valor booleano (TRUE o FALSE):

Operador	Significado	Ejemplo
=	Igual a	departamento = 'Ventas'
<> o !=	Distinto de	cargo <>'Gerente'
>	Mayor que	salario >40000
<	Menor que	salario <35000
>=	Mayor o igual que	salario >= 50000
<=	Menor o igual que	salario <= 48000

Tabla 7: Operadores de comparación en SQL.

```

1  -- Empleados con salario mayor a 45,000
2  SELECT nombre, cargo, salario
3  FROM empleados
4  WHERE salario > 45000;
5
6  -- Empleados que NO son del departamento de TI
7  SELECT nombre, departamento
8  FROM empleados
9  WHERE departamento <> 'TI';
10
11 -- Empleados con salario entre 30,000 y 40,000 (inclusive)
12 SELECT nombre, salario
13 FROM empleados
14 WHERE salario >= 30000
15 AND salario <= 40000;

```

Listing 30: Filtros con operadores de comparación.

8. Modificadores: Filtrado, Ordenamiento y Agrupación

Los **modificadores** son cláusulas y operadores que se combinan con **SELECT** para refinar, filtrar y ordenar los resultados de una consulta. En esta sección cubrimos los modificadores fundamentales que todo programador SQL debe dominar.

8.1. DISTINCT

Por defecto, **SELECT** devuelve **todas** las filas que satisfacen la consulta, incluyendo duplicados. La palabra clave **DISTINCT** instruye al SGBD a eliminar las filas repetidas del resultado, conservando sólo valores únicos.

Definición: DISTINCT

Modificador que se coloca inmediatamente después de **SELECT** y elimina del resultado todas las filas cuyo conjunto de valores en las columnas proyectadas sea idéntico a otra fila ya presente.

```
1  -- ¿Qué departamentos existen en la empresa?
2  -- Sin DISTINCT aparecerán 10 filas (una por empleado)
3  SELECT DISTINCT departamento
4  FROM empleados;
```

Listing 31: DISTINCT sobre una columna.

```
1  -- Combinaciones únicas de departamento + cargo
2  SELECT DISTINCT departamento, cargo
3  FROM empleados
4  ORDER BY departamento, cargo;
```

Listing 32: DISTINCT sobre múltiples columnas.

Nota importante

DISTINCT opera sobre la *combinación completa* de todas las columnas proyectadas, no columna por columna.

8.2. WHERE

La cláusula **WHERE** es el mecanismo de **filtrado por fila** de SQL. Evalúa una condición booleana para cada fila de la tabla fuente; sólo las filas donde la condición resulta **TRUE** pasan al resultado.

Definición: WHERE

Cláusula que restringe las filas devueltas por una consulta a aquellas que satisfacen una expresión booleana dada. Se evalúa *antes* de **GROUP BY**, **HAVING** y **SELECT**.

```
1  -- Empleados del departamento de TI
2  SELECT nombre, cargo, salario
3  FROM    empleados
4  WHERE   departamento = 'TI';
5
6  -- Empleados con salario mayor a 45,000
7  SELECT nombre, salario
8  FROM    empleados
9  WHERE   salario > 45000;
10
11 -- Empleados que ingresaron después del 1 de enero de 2020
12 SELECT nombre, fecha_ingreso
13 FROM    empleados
14 WHERE   fecha_ingreso > '2020-01-01';
```

Listing 33: Filtros básicos con WHERE.

8.3. ORDER BY

ORDER BY define el **criterio de ordenamiento** del resultado final. Sin él, el SGBD no garantiza ningún orden particular entre ejecuciones.

```
1  -- Orden ascendente (A Z, menor mayor): por defecto
2  SELECT nombre, salario
3  FROM    empleados
4  ORDER BY salario ASC;    -- ASC es opcional, es el valor por
                           defecto
5
6  -- Orden descendente (mayor menor)
7  SELECT nombre, salario
8  FROM    empleados
9  ORDER BY salario DESC;
```

Listing 34: Orden ascendente y descendente.

```
1  -- Primero por departamento ( A Z ),
2  -- dentro de cada departamento por salario ( m a y o r menor )
3  SELECT nombre, departamento, salario
4  FROM    empleados
5  ORDER BY departamento ASC,
6  salario    DESC;
```

Listing 35: Ordenamiento compuesto por múltiples columnas.

8.4. LIKE

LIKE permite comparar una columna de texto contra un **patrón**, usando caracteres comodín. Es la herramienta de búsqueda de texto parcial más básica de SQL.

Comodín	Significado	Ejemplo
%	Cero o más caracteres cualesquiera	'Ana%' coincide con Ana, Analista, Anabel
_	Exactamente un carácter cualquiera	'_na' coincide con Ana, Ina, Una

Tabla 8: Comodines del operador LIKE.

```

1  -- Nombres que empiezan con 'A'
2  SELECT nombre FROM empleados
3  WHERE nombre LIKE 'A%';
4
5  -- Nombres que terminan con 'ez'
6  SELECT nombre FROM empleados
7  WHERE nombre LIKE '%ez';
8
9  -- Nombres que contienen 'ar' en cualquier posición
10 SELECT nombre FROM empleados
11 WHERE nombre LIKE '%ar%';
12
13 -- Nombres que NO contienen 'a' (NOT LIKE)
14 SELECT nombre FROM empleados
15 WHERE nombre NOT LIKE '%a%';

```

Listing 36: Patrones con LIKE.

Nota importante

En MySQL, LIKE es **insensible a mayúsculas/minúsculas** por defecto cuando la columna usa un *collation* insensible. Si necesitas distinguir mayúsculas usa LIKE BINARY.

8.5. AND, OR, NOT

Los operadores lógicos permiten **combinar múltiples condiciones** dentro de una cláusula WHERE.

Operador	Resultado TRUE cuando...	Precedencia
NOT	La condición que sigue es FALSE	Mayor (se evalúa primero)
AND	Ambas condiciones son TRUE	Media
OR	Al menos una condición es TRUE	Menor (se evalúa al final)

Tabla 9: Operadores lógicos y su precedencia.

```

1  -- AND: empleados de TI con salario mayor a 47,000
2  SELECT nombre, departamento, salario

```

```
3 FROM    empleados
4 WHERE    departamento = 'TI'
5 AND      salario > 47000;
6
7 -- OR: empleados de Ventas O de Finanzas
8 SELECT  nombre, departamento
9 FROM    empleados
10 WHERE   departamento = 'Ventas'
11 OR      departamento = 'Finanzas';
12
13 -- NOT: empleados que NO son gerentes
14 SELECT  nombre, cargo
15 FROM    empleados
16 WHERE   NOT cargo = 'Gerente';
17 -- equivalente: WHERE cargo <> 'Gerente'
```

Listing 37: Uso de AND, OR y NOT.

8.6. IN y BETWEEN

```
1 -- Empleados en los departamentos Ventas, TI o Finanzas
2 SELECT  nombre, departamento
3 FROM    empleados
4 WHERE   departamento IN ('Ventas', 'TI', 'Finanzas');
5
6 -- Equivalente usando OR (menos legible)
7 WHERE   departamento = 'Ventas'
8 OR      departamento = 'TI'
9 OR      departamento = 'Finanzas';
```

Listing 38: IN para búsquedas en conjuntos.

```
1 -- Empleados con salario entre 40,000 y 50,000 (inclusive)
2 SELECT  nombre, salario
3 FROM    empleados
4 WHERE   salario BETWEEN 40000 AND 50000;
5
6 -- Equivalente usando AND (menos legible)
7 WHERE   salario >= 40000
8 AND     salario <= 50000;
```

Listing 39: BETWEEN para rangos.

8.7. GROUP BY y HAVING

GROUP BY divide las filas en grupos según los valores de una o más columnas. Típicamente se combina con funciones de **agregación** (COUNT, SUM, AVG, MIN, MAX) para calcular resúmenes por grupo.

Definición: GROUP BY

Cláusula que divide las filas del resultado de **WHERE** en grupos, donde cada grupo comparte el mismo valor en las columnas de agrupamiento. Cada grupo produce exactamente **una fila** en el resultado.

```
1  -- N mero de empleados y salario promedio por departamento
2  SELECT departamento,
3  COUNT(*)           AS num_empleados,
4  ROUND(AVG(salario), 2) AS promedio,
5  SUM(salario)       AS masa_salarial,
6  MIN(salario)       AS minimo,
7  MAX(salario)       AS maximo
8  FROM empleados
9  GROUP BY departamento
10 ORDER BY masa_salarial DESC;
```

Listing 40: GROUP BY con funciones de agregación.

HAVING filtra **grupos** (no filas individuales). Es el equivalente de **WHERE** pero aplicado después de que **GROUP BY** ha formado los grupos.

Definición: HAVING

Cláusula que filtra los grupos producidos por **GROUP BY**, conservando sólo aquellos cuya condición (generalmente sobre una función de agregación) es **TRUE**.

```
1  -- Departamentos donde el salario promedio supera 45,000
2  SELECT departamento,
3  ROUND(AVG(salario), 2) AS promedio
4  FROM empleados
5  GROUP BY departamento
6  HAVING AVG(salario) > 45000;
7
8  -- Departamentos con m s de 2 empleados
9  SELECT departamento,
10 COUNT(*) AS num_empleados
11 FROM empleados
12 GROUP BY departamento
13 HAVING COUNT(*) > 2;
```

Listing 41: HAVING: filtrar grupos por condición agregada.

Nota importante

La diferencia entre **WHERE** y **HAVING**: **WHERE** descarta filas *antes* de agrupar (actúa sobre datos individuales); **HAVING** descarta grupos *después* de agrupar (actúa sobre resultados de agregación).

8.8. CASE

CASE es la expresión de **lógica condicional** de SQL: permite evaluar condiciones dentro de una consulta y devolver valores diferentes según cuál se cumpla.

```
1  -- Clasifica el salario en bandas
2  SELECT nombre,
3  salario,
4  CASE
5  WHEN salario >= 55000 THEN 'Banda A (Senior)'
6  WHEN salario >= 45000 THEN 'Banda B (Mid)'
7  WHEN salario >= 35000 THEN 'Banda C (Junior)'
8  ELSE 'Banda D (Entrada)'
9  END AS banda_salarial
10 FROM empleados
11 ORDER BY salario DESC;
```

Listing 42: CASE buscado (evalúa condiciones booleanas).

8.9. COALESCE e IFNULL

Ambas funciones permiten sustituir un valor NULL por un valor alternativo.

```
1  -- Devuelve el primer contacto disponible entre tres
   opciones
2  SELECT nombre,
3  COALESCE(telefono_celular,
4  telefono_fijo,
5  email,
6  'Sin contacto') AS contacto_preferido
7  FROM empleados;
```

Listing 43: COALESCE: devuelve el primer valor no NULL.

Nota importante

IFNULL(a, b) es equivalente a COALESCE(a, b), pero COALESCE acepta cualquier número de argumentos y es parte del estándar SQL. Prefiere COALESCE para mayor portabilidad.

8.10. Combinación de tablas: JOIN

El operador JOIN combina filas de dos o más tablas basándose en una condición de asociación (típicamente igualdad de clave foránea y clave primaria).

8.10.1. INNER JOIN

Definición: INNER JOIN

Devuelve solo las filas donde existe una coincidencia exacta en ambas tablas. Si una fila de la tabla izquierda no tiene par en la tabla derecha, se descarta completamente.

Sintaxis general:

```
1 SELECT columnas
2 FROM tabla_izquierda
3 INNER JOIN tabla_derecha
4 ON tabla_izquierda.clave = tabla_derecha.clave;
```

Listing 44: Sintaxis de INNER JOIN

Ejemplo práctico

Listar clientes que han realizado pedidos:

```
1 SELECT c.nombre ,
2        p.pedido_id ,
3        p.monto
4 FROM clientes AS c
5 INNER JOIN pedidos AS p
6 ON c.cliente_id = p.cliente_id;
```

Listing 45: INNER JOIN: clientes con pedidos

Solo aparecen clientes que han realizado al menos un pedido.

Nota importante

La palabra clave **INNER** es opcional; **JOIN** a secas equivale a **INNER JOIN** en todos los motores SQL principales.

8.10.2. LEFT JOIN

Definición: LEFT JOIN

Devuelve **todas las filas de la tabla izquierda** más las coincidencias de la tabla derecha. Cuando no existe par, las columnas de la tabla derecha se rellenan con **NULL**.

```
1 SELECT columnas
2 FROM tabla_izquierda
3 LEFT JOIN tabla_derecha
4 ON tabla_izquierda.clave = tabla_derecha.clave;
```

Listing 46: Sintaxis de LEFT JOIN

Ejemplo práctico

Listar todos los clientes y, si tienen pedidos, mostrar su monto:

```
1 SELECT c.nombre ,  
2 p.pedido_id ,  
3 p.monto  
4 FROM clientes AS c  
5 LEFT JOIN pedidos AS p  
6 ON c.cliente_id = p.cliente_id;
```

Listing 47: LEFT JOIN: todos los clientes con sus pedidos

Los clientes sin pedidos aparecen con NULL en las columnas de pedidos.

Caso de uso habitual — detectar registros sin par:

```
1 SELECT c.nombre  
2 FROM clientes AS c  
3 LEFT JOIN pedidos AS p  
4 ON c.cliente_id = p.cliente_id  
5 WHERE p.pedido_id IS NULL;
```

Listing 48: Clientes que nunca han realizado un pedido

8.10.3. RIGHT JOIN

Definición: RIGHT JOIN

Devuelve **todas las filas de la tabla derecha** más las coincidencias de la tabla izquierda. Es el simétrico de LEFT JOIN.

Nota importante

En la práctica, RIGHT JOIN se usa con menos frecuencia que LEFT JOIN. Cualquier consulta con RIGHT JOIN puede reescribirse como un LEFT JOIN intercambiando el orden de las tablas:

$$A \text{ RIGHT JOIN } B \equiv B \text{ LEFT JOIN } A$$

8.10.4. UNION

Definición: UNION

Combina verticalmente los resultados de dos o más sentencias SELECT. UNION elimina filas duplicadas; UNION ALL las conserva.

```
1 SELECT columna1, columna2, ...  
2 FROM tabla_a  
3 UNION [ALL]  
4 SELECT columna1, columna2, ...  
5 FROM tabla_b;
```

Listing 49: Sintaxis de UNION / UNION ALL

Ejemplo práctico

Obtener una lista unificada de correos de dos tablas distintas:

```
1 SELECT nombre, correo, 'cliente' AS origen
2 FROM clientes
3 UNION
4 SELECT nombre, correo, 'proveedor' AS origen
5 FROM proveedores
6 ORDER BY nombre;
```

Listing 50: UNION de correos de clientes y proveedores

9. Escritura de Datos

Las sentencias de escritura forman parte del sublenguaje **DML** (*Data Manipulation Language*) de SQL. Permiten insertar, modificar y eliminar filas en las tablas de una base de datos relacional.

Nota importante

Toda operación DML es transaccional. En motores como MySQL/InnoDB o PostgreSQL, los cambios pueden deshacerse con **ROLLBACK** mientras la transacción no haya sido confirmada con **COMMIT**.

9.1. INSERT

Definición: INSERT

Agrega **una o más filas** nuevas a una tabla. Los valores proporcionados deben respetar los tipos de datos, restricciones **NOT NULL** y claves foráneas definidas en el esquema.

Sintaxis — una sola fila:

```
1 INSERT INTO nombre_tabla (col1, col2, col3)
2 VALUES (val1, val2, val3);
```

Listing 51: INSERT de una fila con columnas explícitas

Sintaxis — múltiples filas:

```
1 INSERT INTO nombre_tabla (col1, col2, col3)
2 VALUES
3 (val1a, val2a, val3a),
4 (val1b, val2b, val3b),
5 (val1c, val2c, val3c);
```

Listing 52: INSERT de varias filas en una sola sentencia

Ejemplo práctico

Poblar la tabla **productos** con tres artículos:

```
1 INSERT INTO productos (nombre, precio, stock,
2 categoria_id)
3 VALUES
4 ('Teclado mec nico', 850.00, 40, 3),
5 ('Monitor 24"', 3200.00, 15, 3),
6 ('Silla ergon mica', 5400.00, 8, 5);
```

Listing 53: Inserción múltiple en la tabla productos

Inserción a partir de un **SELECT**:


```
1 INSERT INTO productos_archivo (nombre, precio, categoria_id
  )
2 SELECT nombre, precio, categoria_id
3 FROM   productos
4 WHERE  stock = 0;
```

Listing 54: INSERT ... SELECT: copiar filas desde otra tabla

9.2. UPDATE

Definición: UPDATE

Modifica los valores de **una o más columnas** en las filas que satisfacen la condición WHERE. Si se omite WHERE, la actualización afecta a **todas** las filas de la tabla.

Sintaxis general:

```
1 UPDATE nombre_tabla
2 SET     col1 = nuevo_val1,
3 col2 = nuevo_val2
4 WHERE  condicion;
```

Listing 55: Sintaxis de UPDATE con WHERE

Ejemplo práctico

Aplicar un incremento de 10 % al precio de todos los productos de la categoría 3:

```
1 UPDATE productos
2 SET     precio = precio * 1.10
3 WHERE  categoria_id = 3;
```

Listing 56: UPDATE con expresión aritmética

Nota importante

Buenas prácticas antes de ejecutar un UPDATE:

1. Verificar la condición con un SELECT previo usando el mismo WHERE.
2. Ejecutar dentro de una transacción explícita para poder hacer ROLLBACK si el resultado no es el esperado.
3. En MySQL, activar `sql_safe_updates = 1` para bloquear UPDATE sin WHERE.

9.3. DELETE

Definición: DELETE

Elimina las filas que cumplen la condición **WHERE**. Al igual que **UPDATE**, si se omite **WHERE** se eliminan **todas** las filas de la tabla.

Sintaxis general:

```
1 DELETE FROM nombre_tabla
2 WHERE condicion;
```

Listing 57: Sintaxis de DELETE con WHERE

Ejemplo práctico

Eliminar los productos sin stock de la categoría 5:

```
1 DELETE FROM productos
2 WHERE stock = 0
3 AND categoria_id = 5;
```

Listing 58: DELETE con condición compuesta

9.3.1. Diferencias entre DELETE y TRUNCATE

Tabla 10: Comparación entre DELETE y TRUNCATE

Característica	DELETE	TRUNCATE
Sublenguaje SQL	DML	DDL
Cláusula WHERE	Sí — elimina filas selectivamente	No — elimina todas las filas
Transaccional	Sí — admite ROLLBACK	Depende del motor; en MySQL no
Velocidad	Más lenta (registro fila a fila)	Más rápida (libera páginas de datos)
Reinicio de AUTO_INCREMENT	No — el contador no se reinicia	Sí — el contador vuelve a 1

Nota importante

Ante la duda entre **DELETE** y **TRUNCATE**, preferir **DELETE** dentro de una transacción explícita: ofrece mayor control y trazabilidad.

10. Almacenamiento de Datos Relacionados

Cuando una base de datos posee **integridad referencial** activa, el orden y la forma en que se insertan los registros importan tanto como el contenido mismo. Violar el orden de inserción provoca errores de clave foránea que el motor rechaza antes de escribir cualquier dato.

Nota importante

Regla general: insertar siempre primero la tabla **padre** (la que contiene la clave primaria referenciada) y después la tabla **hijo** (la que contiene la clave foránea). Para eliminar, el orden es el inverso.

10.1. Inserción en Tablas con Relación 1:1

Definición: Relación 1:1

Una fila de la tabla **A** se corresponde con **exactamente una** fila de la tabla **B**, y viceversa. Se implementa colocando la clave foránea en la tabla secundaria con restricción **UNIQUE**.

Esquema de ejemplo:

```

1 CREATE TABLE empleados (
2   empleado_id    INT                UNSIGNED AUTO_INCREMENT PRIMARY
   KEY,
3   nombre         VARCHAR(100) NOT NULL,
4   departamento   VARCHAR(60)  NOT NULL
5 );
6
7 CREATE TABLE expedientes (
8   expediente_id  INT                UNSIGNED AUTO_INCREMENT PRIMARY
   KEY,
9   empleado_id    INT                UNSIGNED NOT NULL UNIQUE,    --
   UNIQUE fuerza 1:1
10  nss             CHAR(11)          NOT NULL,
11  fecha_alta      DATE              NOT NULL,
12  FOREIGN KEY (empleado_id) REFERENCES empleados(empleado_id)
13  ON DELETE CASCADE
14  ON UPDATE CASCADE
15 );

```

Listing 59: Tablas con relación 1:1 — empleado y expediente

```

1 -- Paso 1: insertar en la tabla PADRE
2 INSERT INTO empleados (nombre, departamento)
3 VALUES ('Laura Mendoza', 'Ingeniería'),
4 ('Roberto Solis', 'Finanzas');
5
6 -- Paso 2: insertar en la tabla HIJO referenciando los IDs
   existentes

```

```

7 INSERT INTO expedientes (empleado_id, nss, fecha_alta)
8 VALUES (LAST_INSERT_ID(), '578-12-3456', '2021-03-15');

```

Listing 60: Inserción respetando integridad referencial 1:1

10.2. Inserción en Tablas con Relación 1:N

Definición: Relación 1:N

Una fila de la tabla **padre** puede asociarse con **múltiples** filas de la tabla **hijo**, pero cada fila hijo pertenece a un único padre. Es la cardinalidad más frecuente en bases de datos relacionales.

```

1 -- Paso 1: insertar el cliente (tabla padre)
2 INSERT INTO clientes (nombre, correo)
3 VALUES ('Ana Torres', 'ana.torres@correo.com');
4
5 -- Paso 2: insertar sus pedidos (tabla hijo)
6 INSERT INTO pedidos (cliente_id, fecha, total)
7 VALUES
8 (LAST_INSERT_ID(), '2024-01-15', 1250.00),
9 (LAST_INSERT_ID(), '2024-03-02', 340.50),
10 (LAST_INSERT_ID(), '2024-06-18', 2890.75);

```

Listing 61: Inserción 1:N — un cliente con varios pedidos

10.3. Inserción en Tablas con Relación N:M

Definición: Relación N:M

Múltiples filas de la tabla **A** pueden relacionarse con múltiples filas de la tabla **B**. Se implementa mediante una **tabla intermedia** (también llamada tabla de unión o pivote).

```

1 -- Tabla padre A
2 INSERT INTO estudiantes (nombre)
3 VALUES ('Miguel R os'),
4 ('Sof a Castillo'),
5 ('H ctor N ez');
6
7 -- Tabla padre B
8 INSERT INTO cursos (titulo, credits)
9 VALUES ('Bases de Datos', 6),
10 ('lgebra Lineal', 4),
11 ('Redes de C mputo', 5);

```

Listing 62: Paso 1 — poblar ambas tablas padre

```

1 INSERT INTO inscripciones (estudiante_id, curso_id,
    fecha_alta)

```

```
2 VALUES
3 (1, 1, '2024-08-20'), -- Miguel -> Bases de Datos
4 (1, 3, '2024-08-20'), -- Miguel -> Redes de Computo
5 (2, 1, '2024-08-21'), -- Sofia -> Bases de Datos
6 (2, 2, '2024-08-21'), -- Sofia -> lgebra Lineal
7 (3, 1, '2024-08-22'), -- Hector -> Bases de Datos
8 (3, 2, '2024-08-22'), -- Hector -> lgebra Lineal
9 (3, 3, '2024-08-22'); -- Hector -> Redes de Computo
```

Listing 63: Paso 2 — insertar en la tabla intermedia

Resumen General

Este material ha cubierto los aspectos fundamentales del lenguaje SQL, organizados en torno a tres ejes principales:

1. **Fundamentos teóricos** (secciones 1–4): Historia del problema de los datos, el modelo relacional de Codd, el nacimiento de SQL y la definición conceptual de una base de datos relacional.
2. **Administración de esquemas** (secciones 5–6): Creación y gestión de bases de datos y tablas mediante DDL, incluyendo tipos de datos, restricciones y modificación de estructuras.
3. **Operaciones sobre datos** (secciones 7–9): Consulta (SELECT), escritura (INSERT, UPDATE, DELETE) y manejo de relaciones entre tablas (JOIN, integridad referencial).

El dominio de estos conceptos proporciona la base necesaria para: diseñar esquemas normalizados, escribir consultas eficientes, mantener la integridad de los datos y administrar bases de datos relacionales de forma profesional.